Report

Assignment for the Seminar "Motion Control of Autonomous Robotic Vehicles"

Victor Gabriele De Matteis, University of Trento, 265757

AI use disclosure: Parts of the code, symbolic derivations, and prose in this report were developed with the assistance of Claude (Anthropic), used as an interactive coding and writing aid. All mathematical claims, simulation results, and conclusions were verified and reviewed by the author. *Code repository:* `github.com/ViDem02/MCAV`.

Abstract

We design and verify a Lyapunov-certified geometric cascade controller for a quad UAV tracking a horizontal circular trajectory. The outer position loop is shown to be Globally Asymptotically Stable (GAS) by direct Lyapunov analysis. The inner SO(3) attitude loop requires the gyroscopic cancellation term $\boldsymbol{\Omega} \times J\boldsymbol{\Omega}$ to restore its certificate, following Lee et al. (2010). The combined cascade is GAS provided motor saturation is disabled — a caveat we document and leave as future work. The controller is implemented in MuJoCo via a modular Python framework that supports symbolic verification, headless and video simulation wrappers, and parametric studies. We also include a brief kinematic-level comparison between trajectory tracking and path following showing that trajectory tracking outperforms path following for this specific problem.

Modelling

The aim of this work is twofold: designing a controller for a rotary wing UAV vehicle and simulating its behaviour using the Python Library MuJoCo.

We start by stating the model used as reference. The UAV has four motors positioned in a counter-clockwise direction starting from the front right one. Each motor can be controlled individually by setting its forces f_1, f_2, f_3, f_4. Individual rotor forces produce a wrench $w = [T, \tau_x, \tau_y, \tau_z]^\top$, which stacks together total thrust and torques w.r.t. to the three rotational degrees of freedom.

Additionally, we decide that the front left and back left rotors spin in a counter-clockwise direction and therefore have negative yaw reaction, while front right and back right rotors spin in the opposite direction and therefore impose positive yaw direction.

By Newton's Third Law, the motor exerts an equal and opposite reaction torque on the drone's body, thus Counter-Clockwise rotating motors exert a positive clockwise reaction torque on the frame while clockwise rotating motors exert a negative counter-clockwise reaction torque on the frame. Thus:

$$\tau_z = k_m(-f_1 + f_2 + f_3 - f_4)$$

Roll-torque, which acts with respect to the first rotational degree of freedom, is defined by the cross product of the lever arm and the force vector $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{f}$. For an X-configuration quadcopter, motors 2 and 4 sit on the positive y-axis (left side), while motors 1 and 3 sit on the negative y-axis (right side), at a distance d from the center of mass. We can model this via the following equation:

$$\tau_x = d(-f_1 + f_2 - f_3 + f_4)$$

The reasoning for the pitch torque is quite similar: motors 3 and 4 sit on the negative x-axis, at the rear of the vehicle, and motors 1 and 2 sit at the front. Increasing rear motors (f_3, f_4) pitches the nose down/forward inducing a positive pitch torque while increasing front motors (f_1, f_2) pitches the nose up/backward inducing a negative pitch torque. We model this in the following way:

$$\tau_y = d(-f_1 - f_2 + f_3 + f_4)$$

From the kinematic point of view, the quadcopter is assumed to be a point mass, therefore $\dot{\mathbf{p}} = \mathbf{v}$, the attitude is modelled via a rotation matrix R and the body angular velocities are $\boldsymbol{\Omega} = [\Omega_x, \Omega_y, \Omega_z]^\top$, they are governed by: $\dot{R} = R\hat{\boldsymbol{\Omega}}$ where $\hat{\boldsymbol{\Omega}}$ is the skew-symmetric cross-product matrix.

We assume that the governing differential equation is:

$$m\,\ddot{\mathbf{p}} = mg\,\mathbf{e}_3 - TR\,\mathbf{e}_3$$

where m is the mass of the quadcopter, g is the gravitational acceleration, $\mathbf{e}_3$ is the third standard basis vector, T is the thrust and R is the orientation matrix converting the body frame to the world frame, representing the orientation of the quadcopter.

We model the rotational dynamics using the classical Newton-Euler equations for a rigid body rotating in space:

$$\dot{R} = R\,\hat{\boldsymbol{\Omega}} \qquad J\dot{\boldsymbol{\Omega}} + \boldsymbol{\Omega} \times J\boldsymbol{\Omega} = \boldsymbol{\tau}$$

where J is the 3×3 rigid-body inertia tensor, $\boldsymbol{\Omega}$ is the angular velocity vector in the body-fixed frame, $\boldsymbol{\tau}$ is the net control torque vector applied by the rotors.

Task

The task is for the quadcopter to track a horizontal circle:

$$p_d(t) = \big[R\cos(\omega t),\ R\sin(\omega t),\ z_d\big]^\top,$$

of radius 3.0 m with angular velocity $\omega = 0.5$ rad/s and elevation $z_d = 3$ m.

The aim of the work is to design a controller that is certified to be Globally Asymptotically Stable.

Assumptions

For the controller to be certifiable, the motors cannot have saturation and therefore may produce arbitrary forces, also negative ones. Secondly, this work focuses on simulating a gentle trajectory: we decide to track a circular trajectory with constant angular rate ω, which implies that

$$\dot{\boldsymbol{\Omega}}_d = \mathbf{0}$$

This assumption is required to model a certifiable controller that takes dynamics into account.

Controller

The designed controller is a two-tier geometric cascade. Quadcopters are underactuated—they have 6 degrees of

freedom but only 4 actuators. They cannot translate without first rotating.

To solve this, the controller relies on time-scale separation:

- outer loop (low-bandwidth control): computes desired forces and desired orientations based on position errors;
- inner loop (high-bandwidth control): computes motor torques to track the desired orientation.

The proof of convergence assumes that the inner loop converges so quickly that the outer loop can treat orientation changes as essentially instantaneous.

On the kinematic side, we use a simple proportional control on the position error. We start by considering the position error $\mathbf{e}_p = \mathbf{p} - \mathbf{p}_d$: its time derivative equals $\dot{\mathbf{e}}_p = \mathbf{v} - \mathbf{v}_d$. We want the error to decrease over time. The outer loop controls velocity and therefore assumes complete control over it, so $\dot{\mathbf{x}} - \dot{\mathbf{x}}_d = \mathbf{u} - \mathbf{v}_d$. We choose a control such that $\mathbf{u} - \mathbf{v}_d = -K\,\mathbf{e}_p$.

This is a linear, decoupled, first-order system, therefore each component satisfies $\dot{e}_i = -k_i e_i$, with the known solution $e_i(t) = e_i(0)e^{-k_i t} \to 0$.

Our control $\mathbf{u}$ will be $\mathbf{v}_{\text{cmd}} = \mathbf{v}_d - K\,\mathbf{e}_p$.

The attitude of the vehicle is represented by a rotation matrix $R \in SO(3)$, the Lie group of 3×3 orthogonal matrices with determinant $+1$. The desired attitude is $R_d \in SO(3)$, computed from the desired thrust vector and yaw.

The attitude tracking error is defined on $SO(3)$ as (Lee et al. 2010, eq. 7):

$$\mathbf{e}_R = \frac{1}{2}\,\text{vee}\big(R_d^\top R - R^\top R_d\big) \in \mathbb{R}^3$$

where $\text{vee}(\cdot)$ extracts the axial vector from a skew-symmetric matrix. This error is zero if and only if $R = R_d$.

This form is particularly useful when proving GAS in Lyapunov analysis.

Lyapunov Proof

Symbolic computation

The Lyapunov derivations are carried out symbolically (SymPy) rather than on paper, for three reasons. First, the cancellations involved in the gyroscopic fix — in particular the disappearance of the cross-terms $\mathbf{e}_R^\top \mathbf{e}_\Omega$ from $\dot{V}_2$ — are non-obvious and easy to get wrong by hand. Second, encoding the derivation as code makes it re-runnable: changing the inertia tensor structure or adding a new term re-verifies the certificate automatically. Third, the symbolic output $\dot{V}_2 = -k_R\|\mathbf{e}_R\|^2 - k_w\|\mathbf{e}_\Omega\|^2$ is asserted to be negative semi-definite by an explicit `assert` in `quadcopter_certified_lyapunov.py`, so any future change that breaks the certificate fails loudly rather than silently.

Tier 1 - Kinematic error

The system needs to track a target trajectory $p_d(t)$. We define a simple tracking error $\mathbf{e} = [e_x, e_y, e_z, e_\psi]^\top$. To prove this error shrinks to zero, we define a Lyapunov candidate function that represents something that could be described as the error energy of the position loop:

$$V_1 = \tfrac{1}{2}e_x^2 + \tfrac{1}{2}e_y^2 + \tfrac{1}{2}e_z^2 + \tfrac{1}{2}e_\psi^2$$

For this to be stable, the energy must constantly dissipate. By commanding a velocity $\mathbf{v}_{\text{cmd}} = \mathbf{v}_d - K\,\mathbf{e}$, the system forces the error dynamics to become $\dot{\mathbf{e}} = -K\,\mathbf{e}$.

Evaluating the time derivative of the Lyapunov function ($\dot{V}_1 = \nabla V_1 \cdot \dot{\mathbf{e}}$) gives:

$$\dot{V}_1 = -k_x e_x^2 - k_y e_y^2 - k_z e_z^2 - k_\psi e_\psi^2$$

Because our gain matrix K is positive definite ($k_i > 0$), $\dot{V}_1$ is strictly Negative Definite (ND). The error energy strictly decays to exactly zero. Therefore, the outer loop is Globally Asymptotically Stable (GAS).

Tier 2 - Attitude inner loop

Following Lee et al. (2010, eq. 9), define:

$$V_2(\mathbf{e}_R, \mathbf{e}_\Omega) = k_R\,\Psi(R, R_d) + \tfrac{1}{2}\,\mathbf{e}_\Omega^\top J\,\mathbf{e}_\Omega$$

where $\Psi(R, R_d) = \frac{1}{2}\,\text{tr}(I - R_d^\top R)$ is the attitude tracking function on $SO(3)$, a metric-like quantity that is zero when $R = R_d$ and positive otherwise.

Near the equilibrium, $\Psi \approx \frac{1}{4}\|\mathbf{e}_R\|^2$ (Lee et al. Lemma 1), giving the quadratic approximation:

$$V_2 \approx \tfrac{k_R}{4}\|\mathbf{e}_R\|^2 + \tfrac{1}{2}\,\mathbf{e}_\Omega^\top J\,\mathbf{e}_\Omega$$

Positive Definiteness. Both terms are non-negative; they vanish simultaneously only when $\mathbf{e}_R = \mathbf{0}$ and $\mathbf{e}_\Omega = \mathbf{0}$. Since $k_R > 0$ and $J \succ 0$ (physically), V_2 is positive definite.

Add the gyroscopic term to the torque law (Lee et al. 2010, eq. 16):

$$\boldsymbol{\tau}_{\text{cert}} = -k_R\,\mathbf{e}_R - k_w\,\mathbf{e}_\Omega + \boldsymbol{\Omega} \times J\boldsymbol{\Omega}$$

The angular dynamics become:

$$J\dot{\boldsymbol{\Omega}} = -k_R\,\mathbf{e}_R - k_w\,\mathbf{e}_\Omega + \boldsymbol{\Omega}\times J\boldsymbol{\Omega} - \boldsymbol{\Omega}\times J\boldsymbol{\Omega} = -k_R\,\mathbf{e}_R - k_w\,\mathbf{e}_\Omega$$

The gyroscopic term in the torque *exactly cancels* the gyroscopic term in Euler's equation. The angular error dynamics simplify to:

$$J\dot{\mathbf{e}}_\Omega \approx -k_R\,\mathbf{e}_R - k_w\,\mathbf{e}_\Omega$$

Now the time derivative of V_2 is (Lee et al. 2010, eq. 14):

$$\dot{V}_2\big|_{\text{cert}} = k_R\,\mathbf{e}_R^\top \dot{\mathbf{e}}_R + \mathbf{e}_\Omega^\top J\,\dot{\mathbf{e}}_\Omega$$

Using the SO(3) kinematics $\dot{\mathbf{e}}_R \approx \mathbf{e}_\Omega$ near equilibrium and the simplified angular dynamics:

$$\dot{V}_2\big|_{\text{cert}} = k_R\,\mathbf{e}_R^\top \mathbf{e}_\Omega + \mathbf{e}_\Omega^\top(-k_R\,\mathbf{e}_R - k_w\,\mathbf{e}_\Omega)$$

$$= k_R\,\mathbf{e}_R^\top \mathbf{e}_\Omega - k_R\,\mathbf{e}_\Omega^\top \mathbf{e}_R - k_w\|\mathbf{e}_\Omega\|^2$$

$$= -k_w\|\mathbf{e}_\Omega\|^2 \le 0$$

Negative Semi-Definiteness. $\dot{V}_2\big|_{\text{cert}} \le 0$ (zero only when $\mathbf{e}_\Omega = \mathbf{0}$).

LaSalle for the Attitude Loop Since $\dot{V}_2$ is only negative semi-definite (not ND — it is zero whenever $\mathbf{e}_\Omega = \mathbf{0}$ regardless of $\mathbf{e}_R$), we must invoke LaSalle.

The set where $\dot{V}_2 = 0$:

$$\mathcal{S}_{\text{att}} = \{(\mathbf{e}_R, \mathbf{e}_\Omega) : k_w \|\mathbf{e}_\Omega\|^2 = 0\} = \{(\mathbf{e}_R, \mathbf{0})\}$$

Now we find the largest invariant set $\mathcal{M}_{\text{att}} \subseteq \mathcal{S}_{\text{att}}$. On $\mathcal{S}_{\text{att}}$ we have $\mathbf{e}_\Omega = \mathbf{0}$ and therefore $\dot{\mathbf{e}}_\Omega = \mathbf{0}$. From the angular error dynamics:

$$J\dot{\mathbf{e}}_\Omega = -k_R\,\mathbf{e}_R - k_w\,\mathbf{e}_\Omega = -k_R\,\mathbf{e}_R = \mathbf{0}$$

Since $k_R > 0$, this requires $\mathbf{e}_R = \mathbf{0}$. Therefore:

$$\mathcal{M}_{\text{att}} = \{(\mathbf{0}, \mathbf{0})\}$$

By LaSalle's Invariance Principle, all bounded trajectories converge to $\mathcal{M}_{\text{att}}$.

Combined Lyapunov function

The two tiers are composed by summing the corresponding Lyapunov functions:

$$V_{\text{total}} = V_1 + V_2$$

PD follows directly: the sum of two PD functions is PD, and both contributions are radially unbounded. Computing the time derivative under the certified torque law gives:

$$\dot{V}_{\text{total}} = -\mathbf{e}^\top K\,\mathbf{e} - k_w \|\mathbf{e}_\Omega\|^2$$

This is negative semi-definite, vanishing exactly on $\mathcal{S} = \{\mathbf{e} = \mathbf{0}\} \cap \{\mathbf{e}_\Omega = \mathbf{0}\}$. Applying LaSalle as in Tier 2, the largest invariant set inside $\mathcal{S}$ requires also $\mathbf{e}_R = \mathbf{0}$, so $\mathcal{M} = \{\mathbf{0}\}$.

The full cascade is Globally Asymptotically Stable provided the controller is realisable, i.e. motor forces are not clipped.

Architecture of the coding base

The codebase `mcav_project/` is organised by concern rather than by vehicle, so that the same simulation/-logging/symbolic machinery can be reused for future platforms:

- `src/control/`: cascade controllers (base `CascadeController` and the gyroscopic-fix subclass `CertifiedCascadeController`);
- `src/symbolic/`: SymPy derivations of the three Lyapunov certificates with `assert`-based regression checks;
- `src/simulation/`: `VideoSim`/`HeadlessSim` wrappers around MuJoCo, both exposing an overridable `_loop_step()`;
- `config/`: YAML files for all tunable parameters (gains, trajectory, camera, video FPS) so that experiments are reproducible without touching code;
- `notebooks/`: top-level orchestration and plots.

Two implementation choices deserve mention. The physics engine is MuJoCo (not PyBullet) because of native ARM64 wheels, which gives roughly 2× faster physics on Apple M-series machines and avoids the Rosetta-translation overhead. Video FPS (30) and simulation FPS (500) are kept decoupled: each `_loop_step()` runs at the simulation rate and a separate counter triggers `_capture_frame()` at the video rate. Every step logs a JSON record that the wrapper converts to a pandas DataFrame on demand.

Controller implementation

The controller follows Lee et al. (2010) and operates globally on $SE(3)$, the Lie group of 3D rigid-body positions and rotations. The choice to work directly on rotation matrices — rather than Euler angles — avoids singularities (gimbal lock) and the trigonometric clutter of the alternatives. The architecture is the standard cascade, justified by the time-scale separation between attitude (fast) and position (slow): the outer loop treats the drone as a point mass and computes the body force needed to fix the position error; the inner loop computes the torques needed to tilt the drone so its thrust axis aligns with that force.

Outer loop: translational dynamics

From the position error $\mathbf{e}_p = \mathbf{p} - \mathbf{p}_d$ the outer loop computes a desired body force in three steps. First, a PD law on position produces a velocity command; if the drone is off-target, this commands it to sprint back to the path:

$$\mathbf{v}_{\text{cmd}} = \dot{\mathbf{p}}_d - K_p\,\mathbf{e}_p$$

Next, a PD law on velocity adds the trajectory's analytical feedforward acceleration:

$$\mathbf{a}_{\text{des}} = K_v(\mathbf{v}_{\text{cmd}} - \dot{\mathbf{p}}) + \ddot{\mathbf{p}}_d$$

Finally, the desired force compensates gravity:

$$\mathbf{F}_{\text{des}} = m(\mathbf{a}_{\text{des}} + g\,\mathbf{e}_3)$$

The collective thrust is the magnitude $T = \|\mathbf{F}_{\text{des}}\|$.

Geometric bridge: constructing R_d

Because a quadrotor can only push thrust along its body z-axis, the desired body z-axis must align with $\mathbf{F}_{\text{des}}$. Together with a desired yaw heading ψ_d, this fixes a desired rotation matrix $R_d \in SO(3)$ assembled from three orthonormal columns:

$$\mathbf{z}_d = \frac{\mathbf{F}_{\text{des}}}{\|\mathbf{F}_{\text{des}}\|}, \qquad \mathbf{x}_c = [\cos\psi_d,\ \sin\psi_d,\ 0]^\top$$

$$\mathbf{y}_d = \frac{\mathbf{z}_d \times \mathbf{x}_c}{\|\mathbf{z}_d \times \mathbf{x}_c\|}, \qquad \mathbf{x}_d = \mathbf{y}_d \times \mathbf{z}_d$$

$$R_d = [\,\mathbf{x}_d \ \ \mathbf{y}_d \ \ \mathbf{z}_d\,]$$

The cross product with the yaw-projection vector $\mathbf{x}_c$ gives the lateral axis; the right-hand rule completes the frame as an orthonormal basis.

Inner loop: attitude error on $SO(3)$

The central contribution of Lee et al. is the way the mismatch between R and R_d is measured without subtracting Euler angles. The relative rotation $R_d^\top R$ encodes the attitude error; extracting its skew-symmetric part and applying the vee-map $(\cdot)^\vee$ yields a 3D error vector that lives in the tangent space:

$$\mathbf{e}_R = \tfrac{1}{2}\left(R_d^\top R - R^\top R_d\right)^\vee$$

The angular velocity error has a similar structure but requires a frame conversion — $\boldsymbol{\Omega}_d$ is expressed in the world frame while $\boldsymbol{\Omega}$ is in the body frame, so the desired angular velocity must be pulled into the body frame before subtracting:

$$\mathbf{e}_\Omega = \boldsymbol{\Omega} - R^\top R_d\, \boldsymbol{\Omega}_d$$

A PD torque law then drives both errors to zero:

$$\boldsymbol{\tau} = -k_R\, \mathbf{e}_R - k_\omega\, \mathbf{e}_\Omega$$

This is the "broken" version implemented in the base `CascadeController` — it lacks the $\boldsymbol{\Omega} \times J\boldsymbol{\Omega}$ cancellation needed to make $\dot{V}_2$ strictly negative definite. The `CertifiedCascadeController` subclass overrides only the torque hook:

```
def _compute_torque(self, e_R, e_Omega, state):
    # tau = -kR e_R - kw e_O + Omega x J Omega
    gyro = np.cross(state.angvel, self.J @ state.
        angvel)
    tau  = -self.kR * e_R - self.kw * e_Omega + gyro
    return tau, {"gyro_x": float(gyro[0]),
                 "gyro_y": float(gyro[1]),
                 "gyro_z": float(gyro[2])}
```

The inertia tensor J is read directly from the MuJoCo model via `from_model(cfg, model)` so the cancellation matches the physics exactly — no manual transcription of inertia values that can drift out of sync. The hook returns both the torque and a dict of diagnostics so that the gyroscopic term magnitude is logged at every step without modifying the parent class.

Control allocation (the mixer)

The four virtual commands $[T,\, \tau_x,\, \tau_y,\, \tau_z]^\top$ are mapped to the four physical motor forces $[f_1, f_2, f_3, f_4]^\top$ via the linear allocation matrix A derived in the Modelling section:

$$[f_1,\, f_2,\, f_3,\, f_4]^\top = A^{-1}\,[T,\, \tau_x,\, \tau_y,\, \tau_z]^\top$$

A encodes the X-configuration geometry (arm length d, yaw coefficient k_m, motor spin directions). Since A is constant for a given vehicle, A^{-1} is precomputed once in `__init__()` and applied at every step at negligible cost.

Simulation

Two simulation classes inherit from `VideoSim` and share the same initial condition $\mathbf{p}_0 = [0, 0, 0.5]^\top$ m, level attitude, zero velocities. `QuadcopterCertifiedSim` uses `CertifiedCascadeController`; `QuadcopterDynamicSim` uses the base `CascadeController` (no $\boldsymbol{\Omega} \times J\boldsymbol{\Omega}$). Each `_loop_step()` reads the full state from MuJoCo's `qpos`/`qvel` arrays, evaluates the reference for the current time, calls the controller to produce four motor forces, writes them to `data.ctrl`, and logs the full state plus all controller diagnostics (errors, torques, motor forces, gyroscopic term). Both simulations are run for 60 seconds at 500 Hz, producing 30000 logged samples each.

Path following vs trajectory tracking

We briefly investigated whether path following — which decouples the geometric and temporal objectives via a virtual coordinate γ with $\dot{\gamma} = v_d(1 + k_\gamma\, \hat{\mathbf{t}}^\top \mathbf{e}_p)$ — could give smoother transients than trajectory tracking. The comparison was carried out at the kinematic level (pure NumPy, bypassing MuJoCo dynamics) so as to isolate the outer-loop algorithms.

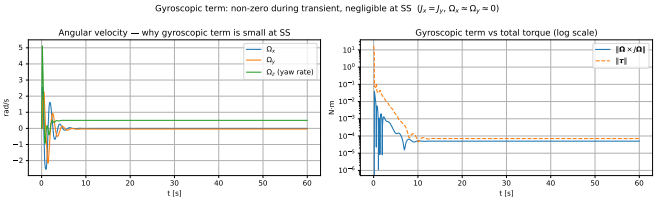
Trajectory tracking consistently outperformed path following for this problem, for four reasons:

- the vehicle starts at $[0, 0, 0.5]$, projecting onto the circle at $\gamma_0 = 0$ with zero along-track error, so the γ correction provides no benefit at $t = 0$;
- the correction term $v_d\, k_\gamma\, e_s$ scales poorly with speed — at high ω the nominal term v_d dominates and the virtual target runs away before the correction acts;
- for a circle, $\dot{\mathbf{p}}_d(t) = R\omega[-\sin\omega t, \cos\omega t, 0]^\top$ is provably the optimal feedforward; path following matches it only when $\dot{\gamma} = \omega$ exactly;
- the cascade inner loop is the real bottleneck at any meaningful speed — path following adds outer-loop complexity without improving inner-loop tracking.

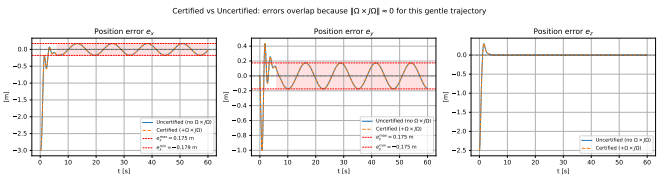
Path following would be beneficial for large cross-track initial offsets at low speed, or in multi-vehicle coordination where timing must be decoupled from geometry. For this problem neither condition holds.

Results

Gyroscopic term magnitude. On a log scale, $\|\boldsymbol{\Omega} \times J\boldsymbol{\Omega}\|$ is 2–3 orders of magnitude below $\|\boldsymbol{\tau}\|$ during the transient and collapses to numerical noise at steady state. The reason is geometric: $J_x = J_y$ in our MJCF model, so the cross products require non-zero Ω_x or Ω_y; at steady state the angular velocity is pure yaw rate ($\Omega_z = \omega = 0.5$ rad/s), making the term vanish.

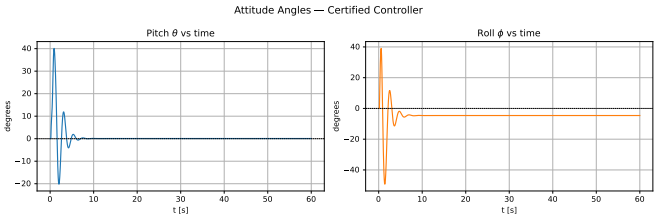


Certified vs uncertified. The two error trajectories overlap visually — expected, since the gyroscopic term is small. The numerical difference is $O(10^{-3})$ m during the transient and floating-point noise thereafter. The theoretical gap is real (without the fix, $\dot{V}_2$ retains cross-terms and is not ND), but for this gentle trajectory the uncertified controller is accidentally well-behaved.



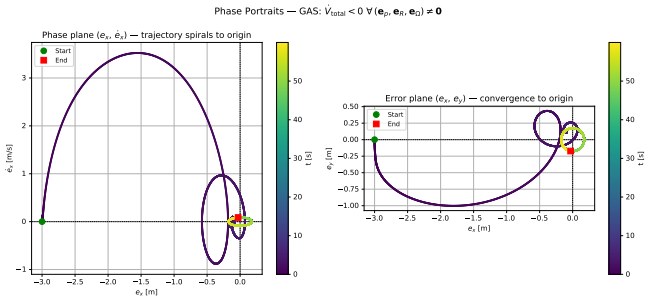
Steady-state error. A residual oscillation of ± 0.18 m in e_x and ± 0.20 m in e_y remains at steady state. This is the UUB (Uniformly Ultimately Bounded) effect from the separation principle: the outer-loop Lyapunov proof assumes ideal velocity tracking, but the inner loop has finite bandwidth and lags slightly behind the centripetal acceleration $R\omega^2 = 0.75$ m/s^2.

Attitude angles. For a circle of radius 3 m at $\omega = 0.5$ rad/s, the required tilt is $\arctan(R\omega^2/g) \approx 4.4°$. Both pitch and roll settle near ± 4–$5°$, matching the analytical prediction.



Motor forces. Without saturation, the controller commands extreme forces during the transient (peaks of ± 300 N), including *negative* thrust — physically impossible on real propellers. This is mathematically consistent with GAS, which requires unbounded control authority. At steady state all four forces converge to hover thrust $f_{\text{hover}} = mg/4 \approx 1.96$ N.

Phase portrait. In the (e_x, e_y) error plane, the trajectory spirals monotonically into the origin, consistent with $\dot{V}_{\text{total}} < 0$ everywhere. The $(e_x, \dot{e}_x)$ phase plane shows the same dissipative spiral structure.



Conclusion

We designed a two-tier geometric cascade controller for a quadcopter on a horizontal circular trajectory and proved it Globally Asymptotically Stable in three steps: the outer kinematic loop is GAS directly by Lyapunov's Direct Method; the inner SO(3) loop becomes GAS after adding the single term $\boldsymbol{\Omega} \times J\boldsymbol{\Omega}$ to the torque law, verified by LaSalle's invariance principle; and the combined cascade is GAS by summing the two Lyapunov functions. Simulation in MuJoCo confirms the predictions: errors converge, attitude angles match the analytical 4.4° tilt, the gyroscopic term is small (but non-zero) during the transient, and the phase portrait shows the monotone convergence to the origin predicted by the certificate.

The main caveat is motor saturation. The GAS certificate requires unbounded control authority, which the transient indeed demands (motor forces peak at ± 300 N, including negative values). Real propellers cannot reproduce this, and clipping the forces degrades GAS to local stability within a Region of Attraction. The natural next step is to handle saturation rigorously via a Control Barrier Function or anti-windup scheme. Other natural extensions include aggressive non-constant-rate trajectories (where the residual inertial feedforward no longer vanishes), asymmetric inertia tensors (where the gyroscopic term becomes dominant rather than small), and multi-vehicle cooperative path following — the natural setting where the path-following decoupling advantage over trajectory tracking would actually matter.